CS 4240: Compilers & Interpreters 11/3/2025

Homework 3: Regular Expressions & Context Free Grammars

Professor Qirun Zhang Due: 11/24/2025 11:59pm

- **We recommend you type your solutions using Overleaf or any other software. Pictures of hand written solutions will be accepted.**
- **All homework must be done independently. You may post clarification questions on Piazza, but be sure to make your post instructor-only if it includes part of your solution.**
- **Late submissions will be accepted within 24 hours of the assignment due date and will receive a 50% point deduction. Any assignments submitted more than 24 hours late will receive a 100% point deduction.**

1 Regular Expressions

(25 points) For each of the languages described below, write a regular expression that matches the language, or explain why no such regular expression exists.

(a.) (10 points) All numerals in base 6 denoting unsigned integers in which successive digits do not increase and there are no unnecessary leading zeros. For example, the following numerals are in the language: 543211, 4310, 22110, 44310, and 0. The following numerals are not in the language: 2512, 1220, 010, 050, and the empty string.

Solution:

```
0
| 1+ 0*
| 2+ 1* 0*
| 3+ 2* 1* 0*
| 4+ 3* 2* 1* 0*
| 5+ 4* 3* 2* 1* 0*
```

(b.) (15 points) A language of program identifiers for a conventional programming language over the alphabet of lowercase and uppercase English letters, digits, and the underscore. An identifier must start with a letter (lower or upper case) or an underscore. If an identifier starts with an underscore, that first symbol must be followed with either a (lower or upper case) letter or a digit. [15 points] For example, the following strings are in the language: `a___0`, `b10`, and `_a_0`. The following strings are not in the language: `2a`, `4_`, and `__a`.

Solution:

```
([a-zA-Z] [a-zA-Z0-9_]*)
| (_ [a-zA-Z0-9] [a-zA-Z0-9_]*)
```

2 Context Free Grammars

(75 points) The following grammar G defines a simple programming language. Each program contains a single function "main" consisting of a sequence of (integer) formal parameters and a single statement, where the statement could be either an assignment or a return statement. Non-terminal symbols start with lower-case English letters; terminal symbols start with upper-case English letters or are punctuation symbols. Note that only non-terminal symbols can occur on the left hand sides of productions.

```
(1) stmt → VAR = expr ;
(2) ret → RETURN expr ;
(3) ret → RETURN ;
(4) expr → term
(5) expr → term op term
(6) op → +
(7) op → -
(8) op → *
(9) op → /
(10) term → VAR
(11) term → CONST
(12) term → ( expr )
(13) prog → MAIN ( params ) { stmt ret }
(14) params → VAR param_tail
(15) params → ε
(16) param_tail → , VAR param_tail
(17) param_tail → ε
```

(a.) (50 points) For each production in G, compute the FIRST set for its right hand side, the FOLLOW set for its left hand side, and the FIRST+ set for the entire production. Please only put terminal symbols or ε in those sets.

Solution:

```
(1) FIRST={VAR}; FOLLOW(stmt)={RETURN}; FIRST+={VAR}
(2) FIRST={RETURN}; FOLLOW(ret)={}; FIRST+={RETURN}
(3) FIRST={RETURN}; FOLLOW(ret)={}; FIRST+={RETURN}
(4) FIRST={VAR,CONST,(}; FOLLOW(expr)={;,)}; FIRST+={VAR,CONST,(}
(5) FIRST={VAR,CONST,(}; FOLLOW(expr)={;,)}; FIRST+={VAR,CONST,(}
(6) FIRST={+}; FOLLOW(op)={VAR,CONST,(}; FIRST+={+}
(7) FIRST={-}; FOLLOW(op)={VAR,CONST,(}; FIRST+={-}
(8) FIRST={*}; FOLLOW(op)={VAR,CONST,(}; FIRST+={*}
(9) FIRST={/}; FOLLOW(op)={VAR,CONST,(}; FIRST+={/}
(10) FIRST={VAR}; FOLLOW(term)={+,-,*,/,;,)}; FIRST+={VAR}
(11) FIRST={CONST}; FOLLOW(term)={+,-,*,/,;,)}; FIRST+={CONST}
(12) FIRST={(}; FOLLOW(term)={+,-,*,/,;,)}; FIRST+={(}
(13) FIRST={MAIN}; FOLLOW(prog)={}; FIRST+={MAIN}
(14) FIRST={VAR}; FOLLOW(params)={)}; FIRST+={VAR}
(15) FIRST={ε}; FOLLOW(params)={)}; FIRST+={)}
(16) FIRST={,}; FOLLOW(param_tail)={)}; FIRST+={,}
(17) FIRST={ε}; FOLLOW(param_tail)={)}; FIRST+={)}
```

(b.) (20 points) Is grammar G an LL(1) grammar? If so, explain why. If not, explain why and provide a grammar G' that is LL(1) and derives exactly the same language as grammar G.

Solution: No, G is not an $LL(1)$ grammar since:

- ret has two productions (2) and (3) with identical FIRST+ = RETURN
- expr has two productions (4) and (5) with identical FIRST+ = VAR, CONST, (.

```
prog       -> MAIN ( params ) { stmt ret }
stmt       -> VAR = expr ;
ret        -> RETURN ret_tail
ret_tail   -> expr ; | ;
expr       -> term expr_tail
expr_tail  -> op term | ε
op         -> + | - | * | /
term       -> VAR | CONST | ( expr )
params     -> VAR param_tail | ε
param_tail -> , VAR param_tail | ε
```

(c.) (5 points) Generalize G to grammar G" that describes programs in which the body of main contains a sequence of one or more statements (`stmt` instances). Grammar G" should be unambiguous.

Solution:

```
prog       -> MAIN ( params ) { stmts ret }
stmts      -> stmt stmts_tail
stmts_tail -> stmt stmts_tail | ε
stmt       -> VAR = expr ;
ret        -> RETURN ret_tail
ret_tail   -> expr ; | ;
expr       -> term expr_tail
expr_tail  -> op term | ε
op         -> + | - | * | /
term       -> VAR | CONST | ( expr )
params     -> VAR param_tail | ε
param_tail -> , VAR param_tail | ε
```